Python Crash Course (Complete Notes)

Python : Part 1

1. Output

Output means displaying information on the screen. Python uses the `print()` function for this.

```
print("Hello, World!")
print("I am learning Python.")
```

Text must be placed inside quotation marks. You can use either double or single quotes:

```
print("Hello", 'Hello') # you can print multiple values by separating them with commas
```

Python can also print numbers and calculations:

```
print(25)
print(10 + 5)
print(8 * 4)
```

Output:

```
25
15
32
```

---

2. Variables

A variable is a named container used to store information.

```
name = "Shradha"
age = 27
marks = 99.5
```

Here:

- `name` stores text.
- `age` stores a whole number.
- `marks` stores a decimal number.
- The `=` symbol assigns a value to a variable.

You can use variables later in the program:

```
name = "Shradha"
age = 27
print("My name is", name)
print("I am", age, "years old.")
```

The value of a variable can change:

```
cgpa = 10
print(cgpa)

cgpa = 20
print(cgpa)
```

Common types of values

```
student_name = "Aman"     # String: text
student_age = 18          # Integer: whole number
percentage = 95           # Float: decimal number
is_present = True         # Boolean: True or False
```

Variable-naming rules

A variable name:

- Can contain letters, numbers and underscores.

- Cannot begin with a number.
- Cannot contain spaces.
- Is case-sensitive: `age` and `Age` are different variables.
- Should clearly describe what it stores.

```
student_name = "Aman"  # Good
studentName = "Aman"   # Valid
x = "Aman"             # Valid, but unclear

# 1name = "Aman"       # Invalid: begins with a number
# student name = "Aman"  # Invalid: contains a space
```

3. Input

Input allows a user to enter information while a program is running. Python uses the `input()` function.

```
name = input("Enter your name: ")
print("Hello,", name)
```

Example:

```
Enter your name: Aarav
Hello, Aarav
```

The message inside `input()` tells the user what to enter.

Input is text by default

The value returned by `input()` is normally a string, even if the user enters a number.

```
age = input("Enter your age: ")
print("You are", age, "years old.")
```

To perform calculations, convert the input into a number:

```
age = int(input("Enter your age: "))
next_age = age + 1

print("Next year, you will be", next_age)
```

Use:

- `int()` for whole numbers.
- `float()` for decimal numbers.

```
price = float(input("Enter the price: "))
quantity = int(input("Enter the quantity: "))

total = price * quantity
print("Total amount:", total)
```

---

4. Comments

Comments are notes written inside a program. Python ignores comments when running the code.

A single-line comment begins with `#`.

```
# Display a welcome message
print("Welcome to Python!")
```

Comments can explain what code does:

```
# Ask the user to enter their name
name = input("Enter your name: ")

# Display a personalised greeting
print("Hello,", name)
```

A short comment can also appear after a statement:

```
age = 18  # Store the student's age
```

Comments are useful for:

- Explaining difficult code.
- Recording important information.
- Making programs easier to understand.
- Temporarily stopping a line from running.

```
print("This line will run.")

# print("This line will not run.")
```

Write comments that explain the purpose of the code, not comments that simply repeat it.

```
# Good: Convert the temperature from Celsius to Fahrenheit
fahrenheit = (celsius * 9 / 5) + 32

# Less helpful: Multiply celsius by 9
```

Practice Exercise 1

Solution

```
# Store Tony's details in variables
first_name = "Tony"
last_name = "Stark"
age = 53
height = 1.85
is_superhero = True

# Ask the user for Tony's superhero name
superhero_name = input("Enter Tony's superhero name: ")

# Print all the details
print("--- Person Details ---")
print("First name:", first_name)
print("Last name:", last_name)
```

```
print("Age:", age)
print("Height:", height, "m")
print("Is a superhero:", is_superhero)
print("Superhero name:", superhero_name)
```

Python : Part 2

1. Type Conversion / Casting

Type conversion changes a value from one data type to another.

```
age = input("What's your age? ")

print(age)
print(type(age))
```

`input()` returns a string, so this produces an error:

```
# print(age + 1)
```

Convert it into an integer before performing a calculation:

```
age = int(input("What's your age? "))

print(age + 1)
```

Common conversion functions:

```
number = 20

print(float(number))  # 20.0
print(str(number))    # "20"
print(bool(number))   # True
```

The value must be suitable for conversion:

```
age = int("18")        # Valid
price = float("9.5")   # Valid

# int("hello")         # Error
```

2. Sum Program

Convert both inputs into numbers before adding them:

```
num1 = int(input("Enter 1st num: "))
num2 = int(input("Enter 2nd num: "))

sum = num1 + num2

print("sum is", sum)
```

Example:

```
Enter the first number: 15
Enter the second number: 10
Sum is 25
```

Avoid using `sum` as a variable name because Python already has a built-in function called `sum()`.

Without `int()`, inputs are joined as strings. So entering `10` and `20` gives `1020`, not `30`.

3. String Methods

```
name = "Tony Stark"
```

`upper()` and `lower()`

```
print(name.upper())  # TONY STARK
print(name.lower())  # tony stark
```

These methods return new strings. They do not change the original:

```
print(name.upper())
print(name)  # Tony Stark
```

Python strings are **immutable**, which means they cannot be changed directly.

To save the new value:

```
name = name.upper()

print(name)
```

`find()`

`find()` returns the position where a character or piece of text begins:

```
name = "Tony Stark"

print(name.find("ark"))  # 7
print(name.find("T"))    # 0
print(name.find("X"))    # -1
```

Python starts counting positions from `0`. A result of `-1` means the text was not found.

Searches are case-sensitive:

```
print(name.find("S"))  # 5
print(name.find("s"))  # -1
```

`replace()`

`replace()` returns a new string with the specified text replaced:

```
name = "Tony Stark"

print(name.replace("Tony Stark", "Iron Man"))
print(name.replace("Stark", "Iron Man"))
print(name.replace("T", "Ph"))
```

Output:

```
Iron Man
Tony Iron Man
Phony Stark
```

The original string remains unchanged unless the result is assigned:

```python
name = name.replace("Tony Stark", "Iron Man")

print(name)
```

4. Keywords

Keywords are reserved words with predefined meanings in Python.

Examples include:

```
if, else, for, while, in, True, False, and, or, not
```

Keywords cannot be used as variable names:

```python
# for = 10  # Invalid
```

The `in` keyword

Use `in` to check whether text is present inside a string:

```python
name = "Tony Stark"

print("S" in name)    # True
print("s" in name)    # False
print("ark" in name)  # True
```

The result is either `True` or `False`, and the check is case-sensitive.

Use `find()` when you need the position:

```
print(name.find("Stark"))  # 5
```

Use `in` when you only need to check whether it exists:

```
print("Stark" in name)  # True
```

Practice Exercise 2

Solution

```
# ------- SOLUTION 1 -------
# Take the prices of three products
price1 = float(input("Enter price of product 1: "))
price2 = float(input("Enter price of product 2: "))
price3 = float(input("Enter price of product 3: "))

# Calculate the total and average
total_bill = price1 + price2 + price3
average_price = total_bill / 3

# Display the results
print("Total bill amount:", total_bill)
print("Average price:", average_price)

# ------- SOLUTION 2 -------
# Take a superhero name and check its first letter
superhero_name = input("Enter a superhero name: ")

starts_with_s = superhero_name.lower().startswith("s")

print("Does the name start with S or s?", starts_with_s)
```

Python : Part 3

1. Arithmetic Operators

Arithmetic operators perform mathematical calculations.

Operator	Meaning	Example	Result
`+`	Addition	`5 + 3`	`8`
`-`	Subtraction	`5 - 3`	`2`
`*`	Multiplication	`5 * 3`	`15`
`/`	Division	`5 / 3`	`1.666...`
`//`	Floor division	`5 // 3`	`1`
`%`	Remainder	`5 % 3`	`2`
`**`	Power	`5 ** 3`	`125`

```
print(5 + 3)
print(5 - 3)
print(5 * 3)
print(5 / 3)
print(5 // 3)
print(5 % 3)
print(5 ** 3)
```

Assignment shortcuts

```
x = 5

x += 3  # x = x + 3
print(x)  # 8
```

Similar shortcuts include:

```
x -= 2
x *= 3
x /= 2
x //= 2
x %= 2
x **= 2
```

2. Operator Precedence

Operator precedence decides which operation Python performs first.

From highest to lowest:

1. Parentheses: `()`
2. Exponentiation: `*`
3. Unary operators: `+x`, `x`
4. Multiplication and division: , `/`, `//`, `%`
5. Addition and subtraction: `+`,
6. Comparisons: `>`, `<`, `>=`, `<=`, `==`, `!=`
7. Logical `not`
8. Logical `and`
9. Logical `or`

```
result = 2 + 3 * 5
print(result)  # 17
```

Multiplication is performed before addition.

Use parentheses to change the order:

```
result = (2 + 3) * 5
print(result)  # 25
```

Operators at the same level are generally evaluated from left to right:

```
print(20 / 5 * 2)  # 8.0
```

Exponentiation is evaluated from right to left:

```
print(2 ** 3 ** 2)  # 2 ** 9 = 512
```

Use parentheses whenever they make an expression clearer.

---

3. Comparison Operators

Comparison operators compare two values and return `True` or `False`.

Operator	Meaning
`>`	Greater than
`<`	Less than
`>=`	Greater than or equal to
`<=`	Less than or equal to
`==`	Equal to
`!=`	Not equal to

```
print(5 > 1)   # True
print(5 < 1)   # False
print(5 >= 1)  # True
print(5 <= 5)  # True
print(5 == 5)  # True
print(5 != 5)  # False
```

`=` assigns a value, while `==` compares two values:

```
age = 18          # Assignment
print(age == 18)  # Comparison
```

---

4. Logical Operators

Logical operators combine or reverse conditions.

`or`

Returns `True` when at least one condition is true:

```
print((5 > 2) or (2 > 5))  # True
```

`and`

Returns `True` only when both conditions are true:

```
print((5 > 2) and (2 > 5))  # False
```

`not`

Reverses a Boolean result:

```
print(not (5 > 2))  # False
```

---

5. Conditional Statements

Conditional statements run code based on whether a condition is `True` or `False`.

`if`

```
age = 20

if age >= 18:
    print("You are an adult.")
    print("You can vote.")
```

A colon `:` is required after the condition. Code inside the block is indented using four spaces.

```
print("End of code")
```

A non-indented statement is outside the `if` block, so it runs in every case.

`if-else`

```
age = 17

if age >= 18:
    print("You can drive.")
else:
    print("You cannot drive.")
```

`if-elif-else`

Use `elif` when there are multiple conditions:

```
age = int(input("Enter your age: "))
```

```
if age > 90:
    print("You are over the allowed age.")
elif age >= 18:
    print("You can drive.")
else:
    print("You cannot drive.")
```

Python checks conditions from top to bottom and runs only the first matching block.

Combining conditions

```
age = 25

if age >= 18 and age <= 90:
    print("You are within the allowed age range.")
else:
    print("You are outside the allowed age range.")
```

Python also supports chained comparisons:

```
if 18 <= age <= 90:
    print("You are within the allowed age range.")
```

Practice Exercise 3

Mini-Project 1 : Calculator

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))

op = input("Enter operation (+, -, *, /, %, ** ): ")

if op == '+':
    print(a + b)
elif op == '-':
    print(a - b)
```

```
elif op == '*':
    print(a * b)
elif op == '/':
    print(a / b)
elif op == '%':
    print(a % b)
else:
    print(a ** b)
```

Python : Part 4

1. `range()` Function

`range()` generates a sequence of numbers, commonly used with loops.

```
numbers = range(5)
print(numbers)  # range(0, 5)
```

The general syntax is:

```
range(start, stop, step)
```

- `start` is included.
- `stop` is excluded.
- `step` controls the difference between numbers.
- The default `start` is `0`.
- The default `step` is `1`.

```
range(5)          # 0, 1, 2, 3, 4
range(1, 6)       # 1, 2, 3, 4, 5
range(2, 11, 2)   # 2, 4, 6, 8, 10
range(5, 0, -1)   # 5, 4, 3, 2, 1
```

The stop value is not included in the sequence.

2. `while` Loop

A `while` loop repeats a block of code while its condition is `True`.

```
i = 1

while i <= 5:
    print(i)
    i += 1
```

Output:

```
1
2
3
4
5
```

Updating the loop variable is important. Without `i += 1`, the condition may always remain true and create an infinite loop.

Increasing star pattern

```
i = 1

while i <= 5:
    print(i * "*")
    i += 1
```

Output:

```
*
**
***
****
*****
```

Decreasing star pattern

```
i = 5

while i > 0:
    print(i * "*")
    i -= 1
```

Output:

```
*****
****
***
**
*
```

3. `for` Loop

A `for` loop repeats code for every value in a sequence.

```
for item in range(5):
    print(item)
```

Output:

```
0
1
2
3
4
```

Print numbers from 1 to 5

```
for i in range(1, 6):
    print(i)
```

Print even numbers from 2 to 10

Use a step of `2`:

```python
for i in range(2, 11, 2):
    print(i)
```

Output:

```
2
4
6
8
10
```

Alternatively, use a condition:

```python
for i in range(2, 11):
    if i % 2 == 0:
        print(i)
```

Use a `for` loop when the number of repetitions is known. Use a `while` loop when repetition depends mainly on a condition.

---

4. `break` and `continue`

`break`

`break` immediately stops the loop.

```python
for i in range(1, 20):
    if i % 7 == 0:
        break

    print(i)
```

Output:

```
1
2
3
4
```

```
5
6
```

The loop stops when it reaches `7`, the first multiple of `7`.

`continue`

`continue` skips the current repetition and moves to the next one.

```python
for i in range(1, 20):
    if i % 7 == 0:
        continue

    print(i)
```

This prints the numbers from `1` to `19`, except multiples of `7`:

```
1
2
3
4
5
6
8
9
10
11
12
13
15
16
17
18
19
```

- `break` stops the entire loop.
- `continue` skips only the current repetition.

Practice Exercise 4

Solution 1 - Print all odd numbers from 1 to 20

```python
for number in range(1, 21, 2):
    print(number)
```

Solution 2 - Print the table of 57

```python
for i in range(1, 11):
    print(57, "x", i, "=", 57 * i)
```

Solution 3 - Print multiples of 3 from 1 to 50, skipping 15

```python
for number in range(3, 51, 3):
    if number == 15:
        continue

    print(number)
```

Solution 4 - Find the first number divisible by both inputs

```python
a = int(input("Enter the first integer: "))
b = int(input("Enter the second integer: "))

for number in range(1, 1001):
    if number % a == 0 and number % b == 0:
        print("First number divisible by both:", number)
        break
```

Python : Part 5

1. List

A list stores multiple values in one variable. It is written using square brackets `[]`.

```
marks = [98, 97, 95, 93.5, "A"]

print(marks)
print(len(marks))  # 5
```

A list can contain values of different data types.

Indexing

List indexing starts from `0`. Negative indexing counts from the end.

```
print(marks[0])   # 98
print(marks[1])   # 97
print(marks[-1])  # A
print(marks[-2])  # 93.5
```

Slicing

Slicing extracts part of a list:

```
print(marks[0:3])    # [98, 97, 95]
print(marks[-3:-1])  # [95, 93.5]
```

The start position is included, while the end position is excluded.

Looping through a list

```
for score in marks:
    print(score)
```

Adding elements

`append()` adds an element at the end:

```
marks.append(95)
print(marks)
```

`insert()` adds an element at a particular position:

```
marks.insert(0, 100)
print(marks)
```

The first argument is the position, and the second is the value.

Checking for an element

```
print(95 in marks)  # True
print(99 in marks)  # False
```

Clearing a list

```
marks.clear()

print(marks)       # []
print(len(marks))  # 0
```

Lists are **mutable**, meaning their elements can be added, removed or changed.

```
marks = [98, 97, 95]
marks[0] = 100

print(marks)  # [100, 97, 95]
```

2. Tuple

A tuple stores multiple values and is written using parentheses `()`.

```
marks = (98, 97, 95, 93, 95, 95)

print(marks)
```

Tuples are **immutable**, so their elements cannot be changed:

```
# marks[0] = 50  # Error
```

`count()`

`count()` returns how many times a value occurs:

```
print(marks.count(95))  # 3
```

`index()`

`index()` returns the position of the first occurrence:

```
print(marks.index(95))  # 2
```

Tuple indexing and looping work like lists:

```
print(marks[0])

for score in marks:
    print(score)
```

---

3. Set

A set stores unique values and is written using curly brackets `{}`.

```
numbers = {4, 5, 4, 5, 9}

print(numbers)
print(len(numbers))  # 3
```

Repeated values are automatically removed.

Sets are unordered, so their display order is not guaranteed. They also do not support indexing:

```
# print(numbers[0])  # Error
```

You can loop through a set:

```
for value in numbers:
    print(value)
```

Elements can be added using `add()`:

```
numbers.add(10)
print(numbers)
```

An empty set must be created using `set()`:

```
empty_set = set()
```

Writing `{}` creates an empty dictionary, not an empty set.

---

4. Dictionary

A dictionary stores information as `key: value` pairs.

```
marks = {
    "Math": 95,
    "Physics": 97,
    "Chemistry": 98
}

print(marks)
```

Each key must be unique.

Accessing a value

Use its key inside square brackets:

```
print(marks["Physics"])  # 97
```

Adding a new pair

```
marks["English"] = 95

print(marks)
```

Updating a value

```
marks["Physics"] = 99

print(marks)
```

Checking for a key

```
print("Math" in marks)     # True
print("Biology" in marks)  # False
```

Looping through a dictionary

```
for subject in marks:
    print(subject, marks[subject])
```

Output:

```
Math 95
Physics 99
Chemistry 98
English 95
```

Dictionaries are mutable, so pairs can be added or updated after creation.

Choosing a Collection

Collection	Written as	Main feature
List	`[1, 2, 3]`	Ordered and changeable
Tuple	`(1, 2, 3)`	Ordered and unchangeable
Set	`{1, 2, 3}`	Stores unique values
Dictionary	`{"name": "Tony"}`	Stores key-value pairs

Practice Exercise 5

Solution 1 - Print unique roll nums

```
roll_numbers = [101, 105, 102, 101, 108, 105, 110]

unique_roll_numbers = set(roll_numbers)

print("Unique roll numbers:", unique_roll_numbers)
```

Solution 2 - Search for an employee using Employee ID

```
employees = [
    (101, "Alice", 50000),
    (102, "Bob", 65000),
    (103, "Charlie", 45000)
]

search_id = int(input("Enter Employee ID: "))
found = False

for employee in employees:
    employee_id = employee[0]

    if employee_id == search_id:
        print("Employee found!")
        print("Employee ID:", employee[0])
        print("Employee Name:", employee[1])
        print("Salary:", employee[2])

        found = True
        break

if found == False:
    print("Employee not found.")
```

Python : Part 6

1. Functions

A function is a reusable block of code that performs a task.

Without a function, the GST calculation must be repeated:

```
price = 100
new_price = price + 0.18 * price

print(new_price)
```

A function is created using the `def` keyword:

```
def add_gst(price):
    new_price = price + 0.18 * price
    print(new_price)
```

Call the function whenever the calculation is needed:

```
add_gst(100)  # 118.0
add_gst(200)  # 236.0
```

Python function names normally use lowercase letters and underscores, such as `add_gst`.

Parameters and arguments

A **parameter** is a variable written in the function definition:

```
def add_gst(price):
    print(price + 0.18 * price)
```

An **argument** is the actual value passed while calling the function:

```
add_gst(100)
```

Here, `price` is the parameter and `100` is the argument.

Multiple parameters

```python
def add_numbers(num1, num2):
    print(num1 + num2)

add_numbers(10, 20)
```

Output:

```
30
```

Returning a value

`return` sends a result back to the place where the function was called.

```python
def add_gst(price):
    new_price = price + 0.18 * price
    return new_price

final_price = add_gst(100)

print("Final price:", final_price)
```

A returned value can be stored or used in another calculation:

```python
price1 = add_gst(100)
price2 = add_gst(200)

total = price1 + price2
print("Total:", total)
```

Function with no parameters

```python
def greet():
    print("Welcome to Python!")

greet()
```

2. Built-in Functions

Built-in functions are available directly in Python and do not require an import.

```
numbers = [1, 2, 3, 4, 5]

print(len(numbers))  # 5
print(max(numbers))  # 5
print(min(numbers))  # 1
print(sum(numbers))  # 15
```

Some commonly used built-in functions are:

Function	Purpose
`print()`	Displays output
`input()`	Takes user input
`len()`	Returns the number of elements
`type()`	Returns the data type
`int()`	Converts a value into an integer
`float()`	Converts a value into a float
`str()`	Converts a value into a string
`max()`	Returns the largest value
`min()`	Returns the smallest value
`sum()`	Returns the total of numeric values

```
print(len("Python"))       # 6
print(str(100))            # "100"
print(type(3.5))           # <class 'float'>
print(sum([10, 20, 30]))   # 60
```

3. Module Functions

A module is a file containing useful functions and other Python code.

Python's `math` module provides mathematical functions.

Importing an entire module

```
import math

print(math.sqrt(16))  # 4.0
print(math.log2(64))  # 6.0
```

When the complete module is imported, use the module name before its function:

```
math.sqrt(16)
```

Importing selected functions

```
from math import sqrt, log2

print(sqrt(16))  # 4.0
print(log2(64))  # 6.0
```

Here, the functions can be used directly because they were imported by name.

Viewing the contents of a module

`dir()` displays the names available inside a module:

```
import math

print(dir(math))
```

Other useful functions and values in the `math` module include:

```
import math

print(math.ceil(4.2))   # 5
print(math.floor(4.8))  # 4
print(math.pow(2, 3))   # 8.0
print(math.pi)          # 3.141592...
```

`math.pi` is a value provided by the module, so it is used without parentheses.

Practice Exercise 6

Solution 1 - Check for Odd or Even

```
def check_odd_even(number):
    if number % 2 == 0:
        print(number, "is even")
    else:
        print(number, "is odd")


check_odd_even(12)
check_odd_even(7)
```

Solution 2 - Count Vowels

```
def count_vowels(text):
    count = 0

    for character in text.lower():
        if character in "aeiou":
            count += 1

    return count


result = count_vowels("Tony Stark")
print("Number of vowels:", result)
```

Solution 3 - Check whether number is prime or not

```
def check_prime(number):
    if number <= 1:
        print(number, "is not prime")
        return

    for i in range(2, number):
```

```
        if number % i == 0:
            print(number, "is not prime")
            return

    print(number, "is prime")


check_prime(7)
check_prime(10)
```

Solution 4 - Return average marks

```
def calculate_average(marks):
    if len(marks) == 0:
        return 0

    average = sum(marks) / len(marks)
    return average


student_marks = [85, 90, 78, 92, 80]

result = calculate_average(student_marks)
print("Average marks:", result)
```

Mini-Project 2 : Guessing Game

```
import random
lucky_num = random.randint(1, 50)

while True:
    user_num = int(input("Guess the lucky number: "))

    if lucky_num == user_num:
```

```
            print("Correct Guess, YOU WON!!")
            break
        elif lucky_num < user_num:
            print("Too High")
        else:
            print("Too Low")

print("---------- Game has ended----------")
```

Hope you enjoying learning Python via this Crash Course.

Next Steps : Start learning some of the more advances Python concepts in detail with:

ApnaCollege's Detailed Python Series: Shradha Khapra Python Language Full Course (2026)

SQL

(Notes by Apna College)

What is Database?
Database is a collection of interrelated data.

What is DBMS?
DBMS (*Database Management System*) is software used to create, manage, and organize databases.

What is RDBMS?

- RDBMS (Relational *Database Management System)* - is a DBMS based on the concept of tables (also called relations).
- Data is organized into tables (also known as relations) with rows (records) and columns (attributes).
- Eg - MySQL, PostgreSQL, Oracle etc.

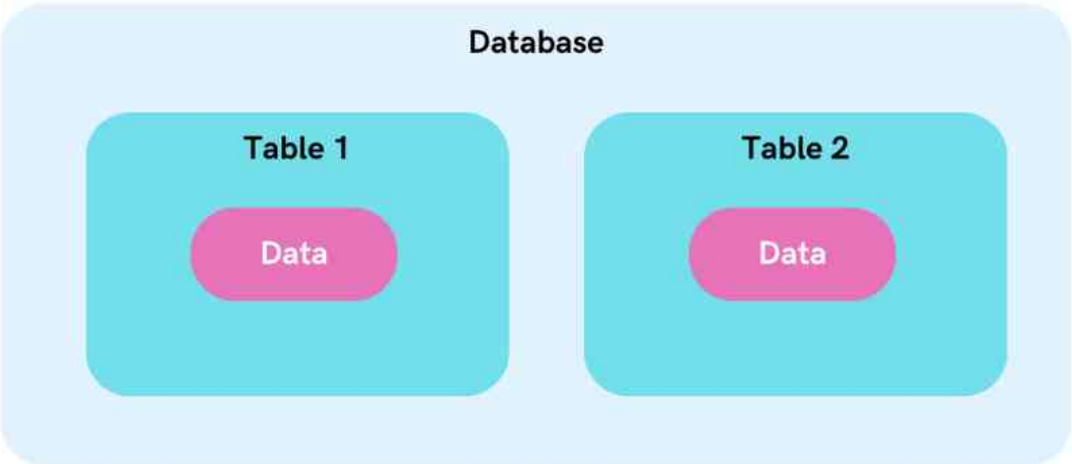
What is SQL?
SQL is *Structured Query Language* - used to store, manipulate and retrieve data from RDBMS.
(It is not a database, it is a language used to interact with database)

We use SQL for *CRUD* Operations :

- **CREATE** - To create databases, tables, insert tuples in tables etc
- **READ** - To read data present in the database.
- **UPDATE** - Modify already inserted data.
- **DELETE** - Delete database, table or specific data point/tuple/row or multiple rows.

*Note - SQL keywords are NOT case sensitive. Eg: select is the same as SELECT in SQL.

SQL v/s MySQL
SQL is a language used to perform CRUD operations in Relational DB, while MySQL is a RDBMS that uses SQL.



SQL Data Types
In SQL, data types define the kind of data that can be stored in a column or variable.

To See all data types of MYSQL, visit :
https://dev.mysql.com/doc/refman/8.0/en/data-types.html

Here are the frequently used SQL data types:

DATATYPE	DESCRIPTION	USAGE
CHAR	string(0-255), can store characters of fixed length	CHAR(50)
VARCHAR	string(0-255), can store characters up to given length	VARCHAR(50)
BLOB	string(0-65535), can store binary large object	BLOB(1000)
INT	integer(-2,147,483,648 to 2,147,483,647)	INT
TINYINT	integer(-128 to 127)	TINYINT
BIGINT	integer(-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807)	BIGINT
BIT	can store x-bit values. x can range from 1 to 64	BIT(2)
FLOAT	Decimal number - with precision to 23 digits	FLOAT
DOUBLE	Decimal number - with 24 to 53 digits	DOUBLE
BOOLEAN	Boolean values 0 or 1	BOOLEAN
DATE	date in format of YYYY-MM-DD ranging from 1000-01-01 to 9999-12-31	DATE
TIME	HH:MM:SS	TIME
YEAR	year in 4 digits format ranging from 1901 to 2155	YEAR

*Note - CHAR is for fixed length & VARCHAR is for variable length strings. Generally, VARCHAR is better as it only occupies necessary memory & works more efficiently.

We can also use UNSIGNED with datatypes when we only have positive values to add.
Eg - UNSIGNED INT

Types of SQL Commands:

1. **DQL** *(Data Query Language)* : Used to retrieve data from databases. (SELECT)

2. **DDL** *(Data Definition Language)* : Used to create, alter, and delete database objects like tables, indexes, etc. (CREATE, DROP, ALTER, RENAME, TRUNCATE)

3. **DML** *(Data Manipulation Language)*: Used to modify the database. (INSERT, UPDATE, DELETE)

4. **DCL** *(Data Control Language)*: Used to grant & revoke permissions. (GRANT, REVOKE)

5. **TCL** *(Transaction Control Language)*: Used to manage transactions. (COMMIT, ROLLBACK, START TRANSACTIONS, SAVEPOINT)

---

1. Data Definition Language (DDL)

Data Definition Language (DDL) is a subset of SQL (Structured Query Language) responsible for defining and managing the structure of databases and their objects.

DDL commands enable you to create, modify, and delete database objects like tables, indexes, constraints, and more.

Key DDL Commands are:

- **CREATE TABLE:**
    - Used to create a new table in the database.
    - Specifies the table name, column names, data types, constraints, and more.
    - Example:
      CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(50), salary DECIMAL(10, 2));

- **ALTER TABLE:**
    - Used to modify the structure of an existing table.
    - You can add, modify, or drop columns, constraints, and more.
    - Example: ALTER TABLE employees ADD COLUMN email VARCHAR(100);

- **DROP TABLE:**
    - Used to delete an existing table along with its data and structure.
    - Example: DROP TABLE employees;

- **CREATE INDEX:**
    - Used to create an index on one or more columns in a table.
    - Improves query performance by enabling faster data retrieval.
    - Example: CREATE INDEX idx_employee_name ON employees (name);
- **DROP INDEX:**
    - Used to remove an existing index from a table.
    - Example: DROP INDEX idx_employee_name;
- **CREATE CONSTRAINT:**
    - Used to define constraints that ensure data integrity.
    - Constraints include PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK.
    - Example: ALTER TABLE orders ADD CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES customers(id);
- **DROP CONSTRAINT:**
    - Used to remove an existing constraint from a table.
    - Example: ALTER TABLE orders DROP CONSTRAINT fk_customer;
- **TRUNCATE TABLE:**
    - Used to delete the data inside a table, but not the table itself.
    - Syntax – TRUNCATE TABLE table_name

2. DATA QUERY/RETRIEVAL LANGUAGE (DQL or DRL)

DQL (Data Query Language) is a subset of SQL focused on retrieving data from databases.

The SELECT statement is the foundation of DQL and allows us to extract specific columns from a table.

- **SELECT:**

The SELECT statement is used to select data from a database.

Syntax: SELECT column1, column2, ... FROM table_name;

Here, column1, column2, ... are the field names of the table.

If you want to select all the fields available in the table, use the following syntax:
SELECT * FROM table_name;

Ex: SELECT CustomerName, City FROM Customers;

- **WHERE:**

The WHERE clause is used to filter records.

Syntax: SELECT column1, column2, ... FROM table_name WHERE condition;

Ex: SELECT * FROM Customers WHERE Country='Mexico';

Operators used in WHERE are:

= : Equal
> : Greater than
< : Less than
>= : Greater than or equal
<= : Less than or equal
<> : Not equal.

Note: In some versions of SQL this operator may be written as !=

- **AND, OR and NOT:**

- The WHERE clause can be combined with AND, OR, and NOT operators.
- The AND and OR operators are used to filter records based on more than one condition:
- The AND operator displays a record if all the conditions separated by AND are TRUE.
- The OR operator displays a record if any of the conditions separated by OR is TRUE.
- The NOT operator displays a record if the condition(s) is NOT TRUE.

Syntax:

SELECT column1, column2, ... FROM table_name WHERE condition1 AND condition2 AND condition3 ...;

SELECT column1, column2, ... FROM table_name WHERE condition1 OR condition2 OR condition3 ...;

SELECT column1, column2, ... FROM table_name WHERE NOT condition;

Example:

SELECT * FROM Customers WHERE Country='India' AND City='Japan';

SELECT * FROM Customers WHERE Country='America' AND (City='India' OR City='Korea');

- **DISTINCT:**

Removes duplicate rows from query results.

Syntax: SELECT DISTINCT column1, column2 FROM table_name;

- **LIKE**:

The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

There are two wildcards often used in conjunction with the LIKE operator:

- The percent sign (%) represents zero, one, or multiple characters
- The underscore sign (_) represents one, single character

Example: SELECT * FROM employees WHERE first_name LIKE 'J%';

WHERE CustomerName LIKE 'a%'
- Finds any values that start with "a"

WHERE CustomerName LIKE '%a'
- Finds any values that end with "a"

WHERE CustomerName LIKE '%or%'
- Finds any values that have "or" in any position

WHERE CustomerName LIKE '_r%'
- Finds any values that have "r" in the second position

WHERE CustomerName LIKE 'a_%'

- Finds any values that start with "a" and are at least 2 characters in length

WHERE CustomerName LIKE 'a__%'

- Finds any values that start with "a" and are at least 3 characters in length

WHERE ContactName LIKE 'a%o'

- Finds any values that start with "a" and ends with "o"

- **IN:**

Filters results based on a list of values in the WHERE clause.

Example: SELECT * FROM products WHERE category_id IN (1, 2, 3);

- **BETWEEN**:

Filters results within a specified range in the WHERE clause.

Example: SELECT * FROM orders WHERE order_date BETWEEN '2023-01-01' AND '2023-06-30';

- **IS NULL**:

Checks for NULL values in the WHERE clause.

Example: SELECT * FROM customers WHERE email IS NULL;

- **AS**:

Renames columns or expressions in query results.

Example: SELECT first_name AS "First Name", last_name AS "Last Name" FROM employees;

- **ORDER BY**

The ORDER BY clause allows you to sort the result set of a query based on one or more columns.

Basic Syntax:

- The ORDER BY clause is used after the SELECT statement to sort query results.

- Syntax: SELECT column1, column2 FROM table_name ORDER BY column1 [ASC|DESC];

Ascending and Descending Order:

- By default, the ORDER BY clause sorts in ascending order (smallest to largest).
- You can explicitly specify descending order using the DESC keyword.
- Example: SELECT product_name, price FROM products ORDER BY price DESC;

Sorting by Multiple Columns:

- You can sort by multiple columns by listing them sequentially in the ORDER BY clause.
- Rows are first sorted based on the first column, and for rows with equal values, subsequent columns are used for further sorting.
- Example: SELECT first_name, last_name FROM employees ORDER BY last_name, first_name;

Sorting by Expressions:

- It's possible to sort by calculated expressions, not just column values.
- Example: SELECT product_name, price, price * 1.1 AS discounted_price FROM products ORDER BY discounted_price;

Sorting NULL Values:

- By default, NULL values are considered the smallest in ascending order and the largest in descending order.
- You can control the sorting behaviour of NULL values using the NULLS FIRST or NULLS LAST options.
- Example: SELECT column_name FROM table_name ORDER BY column_name NULLS LAST;

Sorting by Position:

- Instead of specifying column names, you can sort by column positions in the ORDER BY clause.
- Example: SELECT product_name, price FROM products ORDER BY 2 DESC, 1 ASC;

- **GROUP BY**

The GROUP BY clause in SQL is used to group rows from a table based on one or more columns.

Syntax:

- The GROUP BY clause follows the SELECT statement and is used to group rows based on specified columns.

- Syntax: SELECT column1, aggregate_function(column2) FROM table_name GROUP BY column1;

- Aggregation Functions:
    - Aggregation functions (e.g., COUNT, SUM, AVG, MAX, MIN) are often used with GROUP BY to calculate values for each group.
    - Example: SELECT department, AVG(salary) FROM employees GROUP BY department;
- Grouping by Multiple Columns:

    - You can group by multiple columns by listing them in the GROUP BY clause.
    - This creates a hierarchical grouping based on the specified columns.
    - Example: SELECT department, gender, AVG(salary) FROM employees GROUP BY department, gender;

- HAVING Clause:

    - The HAVING clause is used with GROUP BY to filter groups based on aggregate function results.
    - It's similar to the WHERE clause but operates on grouped data.
    - Example: SELECT department, AVG(salary) FROM employees GROUP BY department HAVING AVG(salary) > 50000;

- Combining GROUP BY and ORDER BY:

    - You can use both GROUP BY and ORDER BY in the same query to control the order of grouped results.
    - Example: SELECT department, COUNT(*) FROM employees GROUP BY department ORDER BY COUNT(*) DESC;

- **AGGREGATE FUNCTIONS**

These are used to perform calculations on groups of rows or entire result sets. They provide insights into data by summarising and processing information.

Common Aggregate Functions:

- COUNT():
  Counts the number of rows in a group or result set.

- SUM():
  Calculates the sum of numeric values in a group or result set.

- AVG():

Computes the average of numeric values in a group or result set.

- MAX():
  Finds the maximum value in a group or result set.

- MIN():
  Retrieves the minimum value in a group or result set.

3. DATA MANIPULATION LANGUAGE

Data Manipulation Language (DML) in SQL encompasses commands that manipulate data within a database. DML allows you to insert, update, and delete records, ensuring the accuracy and currency of your data.

- **INSERT:**

- The INSERT statement adds new records to a table.
- Syntax: INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);

- Example: INSERT INTO employees (first_name, last_name, salary) VALUES ('John', 'Doe', 50000);

- **UPDATE**:

- The UPDATE statement modifies existing records in a table.
- Syntax: UPDATE table_name SET column1 = value1, column2 = value2, ... WHERE condition;
- Example: UPDATE employees SET salary = 55000 WHERE first_name = 'John';

- **DELETE**:

- The DELETE statement removes records from a table.
- Syntax: DELETE FROM table_name WHERE condition;
- Example: DELETE FROM employees WHERE last_name = 'Doe';

4. Data Control Language (DCL)

Data Control Language focuses on the management of access rights, permissions, and security-related aspects of a database system.

DCL commands are used to control who can access the data, modify the data, or perform administrative tasks within a database.

DCL is an important aspect of database security, ensuring that data remains protected and only authorised users have the necessary privileges.

There are two main DCL commands in SQL: GRANT and REVOKE.

1. GRANT:

The GRANT command is used to provide specific privileges or permissions to users or roles. Privileges can include the ability to perform various actions on tables, views, procedures, and other database objects.

Syntax:

```
GRANT privilege_type
ON object_name
TO user_or_role;
```

In this syntax:

- privilege_type refers to the specific privilege or permission being granted (e.g., SELECT, INSERT, UPDATE, DELETE).
- object_name is the name of the database object (e.g., table, view) to which the privilege is being granted.
- user_or_role is the name of the user or role that is being granted the privilege.

Example: Granting SELECT privilege on a table named "Employees" to a user named "Analyst":

```
GRANT SELECT ON Employees TO Analyst;
```

2. REVOKE:

The REVOKE command is used to remove or revoke specific privileges or permissions that have been previously granted to users or roles.

Syntax:

```
REVOKE privilege_type
ON object_name
```

FROM user_or_role;

In this syntax:

- privilege_type is the privilege or permission being revoked.
- object_name is the name of the database object from which the privilege is being revoked.
- user_or_role is the name of the user or role from which the privilege is being revoked.

Example: Revoking the SELECT privilege on the "Employees" table from the "Analyst" user:

REVOKE SELECT ON Employees FROM Analyst;

DCL and Database Security:

DCL plays a crucial role in ensuring the security and integrity of a database system.

By controlling access and permissions, DCL helps prevent unauthorised users from tampering with or accessing sensitive data. Proper use of GRANT and REVOKE commands ensures that only users who require specific privileges can perform certain actions on database objects.

5. Transaction Control Language (TCL)

Transaction Control Language (TCL) deals with the management of transactions within a database.
TCL commands are used to control the initiation, execution, and termination of transactions, which are sequences of one or more SQL statements that are executed as a single unit of work.
Transactions ensure data consistency, integrity, and reliability in a database by grouping related operations together and either committing or rolling back changes based on the success or failure of those operations.

There are three main TCL commands in SQL: COMMIT, ROLLBACK, and SAVEPOINT.

1. COMMIT:

The COMMIT command is used to permanently save the changes made during a transaction.

It makes all the changes applied to the database since the last COMMIT or ROLLBACK command permanent.
Once a COMMIT is executed, the transaction is considered successful, and the changes are made permanent.

Example: Committing changes made during a transaction:

```
UPDATE Employees
SET Salary = Salary * 1.10
WHERE Department = 'Sales';

COMMIT;
```

2. ROLLBACK:

The ROLLBACK command is used to undo changes made during a transaction.
It reverts all the changes applied to the database since the transaction began.

ROLLBACK is typically used when an error occurs during the execution of a transaction, ensuring that the database remains in a consistent state.

Example: Rolling back changes due to an error during a transaction:

```
BEGIN;

UPDATE Inventory
SET Quantity = Quantity - 10
WHERE ProductID = 101;

-- An error occurs here

ROLLBACK;
```

3. SAVEPOINT:

The SAVEPOINT command creates a named point within a transaction, allowing you to set a point to which you can later ROLLBACK if needed.

SAVEPOINTs are useful when you want to undo part of a transaction while preserving other changes.

Syntax: SAVEPOINT savepoint_name;

Example: Using SAVEPOINT to create a point within a transaction:

```
BEGIN;
```

```
UPDATE Accounts
SET Balance = Balance - 100
WHERE AccountID = 123;

SAVEPOINT before_withdrawal;

UPDATE Accounts
SET Balance = Balance + 100
WHERE AccountID = 456;

-- An error occurs here

ROLLBACK TO before_withdrawal;

-- The first update is still applied

COMMIT;
```

TCL and Transaction Management:

Transaction Control Language (TCL) commands are vital for managing the integrity and consistency of a database's data.
They allow you to group related changes into transactions, and in the event of errors, either commit those changes or roll them back to maintain data integrity.
TCL commands are used in combination with Data Manipulation Language (DML) and other SQL commands to ensure that the database remains in a reliable state despite unforeseen errors or issues.

JOINS

In a DBMS, a join is an operation that combines rows from two or more tables based on a related column between them.
Joins are used to retrieve data from multiple tables by linking them together using a common key or column.

Types of Joins:

1. Inner Join
2. Outer Join
3. Cross Join
4. Self Join

1) Inner Join

An inner join combines data from two or more tables based on a specified condition, known as the join condition.
The result of an inner join includes only the rows where the join condition is met in all participating tables.
It essentially filters out non-matching rows and returns only the rows that have matching values in both tables.

Syntax:

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

Here:

- columns refers to the specific columns you want to retrieve from the tables.
- table1 and table2 are the names of the tables you are joining.
- column is the common column used to match rows between the tables.
- The ON clause specifies the join condition, where you define how the tables are related.

Example: Consider two tables: Customers and Orders.

Customers Table:

CustomerID	CustomerName
1	Alice
2	Bob
3	Carol

Orders Table:

OrderID	CustomerID	Product

101	1	Laptop
102	3	Smartphone
103	2	Headphones

Inner Join Query:

```
SELECT Customers.CustomerName, Orders.Product
FROM Customers
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerName	Product
Alice	Laptop
Bob	Headphones
Carol	Smartphone

2) Outer Join

Outer joins combine data from two or more tables based on a specified condition, just like inner joins. However, unlike inner joins, outer joins also include rows that do not have matching values in both tables.
Outer joins are particularly useful when you want to include data from one table even if there is no corresponding match in the other table.

Types:

There are three types of outer joins: left outer join, right outer join, and full outer join.

1. Left Outer Join (Left Join):

A left outer join returns all the rows from the left table and the matching rows from the right table.

If there is no match in the right table, the result will still include the left table's row with NULL values in the right table's columns.

Example:

```
SELECT Customers.CustomerName, Orders.Product
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerName	Product
Alice	Laptop
Bob	Headphones
Carol	Smartphone
NULL	Monitor

In this example, the left outer join includes all rows from the Customers table.

Since there is no matching customer for the order with OrderID 103 (Monitor), the result includes a row with NULL values in the CustomerName column.

2. Right Outer Join (Right Join):

A right outer join is similar to a left outer join, but it returns all rows from the right table and the matching rows from the left table.

If there is no match in the left table, the result will still include the right table's row with NULL values in the left table's columns.

Example: Using the same Customers and Orders tables.

```
SELECT Customers.CustomerName, Orders.Product
FROM Customers
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerName	Product
Alice	Laptop
Carol	Smartphone
Bob	Headphones
NULL	Keyboard

Here, the right outer join includes all rows from the Orders table. Since there is no matching order for the customer with CustomerID 4, the result includes a row with NULL values in the CustomerName column.

3. Full Outer Join (Full Join):

A full outer join returns all rows from both the left and right tables, including matches and non-matches.

If there's no match, NULL values appear in columns from the table where there's no corresponding value.

Example: Using the same Customers and Orders tables.

```
SELECT Customers.CustomerName, Orders.Product
FROM Customers
FULL OUTER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerName	Product
Alice	Laptop
Bob	Headphones
Carol	Smartphone

NULL	Monitor
NULL	Keyboard

In this full outer join example, all rows from both tables are included in the result. Both non-matching rows from the Customers and Orders tables are represented with NULL values.

3) Cross Join

A cross join, also known as a Cartesian product, is a type of join operation in a Database Management System (DBMS) that combines every row from one table with every row from another table.

Unlike other join types, a cross join does not require a specific condition to match rows between the tables. Instead, it generates a result set that contains all possible combinations of rows from both tables.

Cross joins can lead to a large result set, especially when the participating tables have many rows.

Syntax:

```
SELECT columns
FROM table1
CROSS JOIN table2;
```

In this syntax:

- columns refers to the specific columns you want to retrieve from the cross-joined tables.
- table1 and table2 are the names of the tables you want to combine using a cross join.

Example: Consider two tables: Students and Courses.

Students Table:

StudentID	StudentName
1	Alice

2	Bob

Courses Table:

CourseID	CourseName
101	Maths
102	Science

Cross Join Query:

```
SELECT Students.StudentName, Courses.CourseName
FROM Students
CROSS JOIN Courses;
```

Result:

StudentName	CourseName
Alice	Maths
Alice	Science
Bob	Maths
Bob	Science

In this example, the cross join between the Students and Courses tables generates all possible combinations of rows from both tables. As a result, each student is paired with each course, leading to a total of four rows in the result set.

4) Self Join

A self join involves joining a table with itself.

This technique is useful when a table contains hierarchical or related data and you need to compare or analyse rows within the same table.

Self joins are commonly used to find relationships, hierarchies, or patterns within a single table.

In a self join, you treat the table as if it were two separate tables, referring to them with different aliases.

Syntax:

The syntax for performing a self join in SQL is as follows:

```
SELECT columns
FROM table1 AS alias1
JOIN table1 AS alias2 ON alias1.column = alias2.column;
```

In this syntax:

- columns refers to the specific columns you want to retrieve from the self-joined table.
- table1 is the name of the table you're joining with itself.
- alias1 and alias2 are aliases you assign to the table instances for differentiation.
- column is the column you use as the join condition to link rows from the same table.

Example: Consider an Employees table that contains information about employees and their managers.

Employees Table:

EmployeeID	EmployeeName	ManagerID
1	Alice	3
2	Bob	3
3	Carol	NULL
4	David	1

Self Join Query:

```
SELECT e1.EmployeeName AS Employee, e2.EmployeeName AS Manager
FROM Employees AS e1
JOIN Employees AS e2 ON e1.ManagerID = e2.EmployeeID;
```

Result:

Employee	Manager

Alice	Carol
Bob	Carol
David	Alice

In this example, the self join is performed on the Employees table to find the relationship between employees and their managers. The join condition connects the ManagerID column in the e1 alias (representing employees) with the EmployeeID column in the e2 alias (representing managers).

SET OPERATIONS

Set operations in SQL are used to combine or manipulate the result sets of multiple SELECT queries.
They allow you to perform operations similar to those in set theory, such as union, intersection, and difference, on the data retrieved from different tables or queries.

Set operations provide powerful tools for managing and manipulating data, enabling you to analyse and combine information in various ways.

There are four primary set operations in SQL:

- UNION
- INTERSECT
- EXCEPT (or MINUS)
- UNION ALL

1. UNION:

The UNION operator combines the result sets of two or more SELECT queries into a single result set.
It removes duplicates by default, meaning that if there are identical rows in the result sets, only one instance of each row will appear in the final result.

Example:

Assume we have two tables: Customers and Suppliers.

Customers Table:

CustomerID	CustomerName
1	Alice
2	Bob

Suppliers Table:

SupplierID	SupplierName
101	SupplierA
102	SupplierB

UNION Query:

```
SELECT CustomerName FROM Customers
UNION
SELECT SupplierName FROM Suppliers;
```

Result:

CustomerName
Alice
Bob
SupplierA
SupplierB

2. INTERSECT:

The INTERSECT operator returns the common rows that exist in the result sets of two or more SELECT queries.

It only returns distinct rows that appear in all result sets.

Example: Using the same tables as before.

```
SELECT CustomerName FROM Customers
INTERSECT
SELECT SupplierName FROM Suppliers;
```

Result:

CustomerName

In this example, there are no common names between customers and suppliers, so the result is an empty set.

3. EXCEPT (or MINUS):

The EXCEPT operator (also known as MINUS in some databases) returns the distinct rows that are present in the result set of the first SELECT query but not in the result set of the second SELECT query.

Example: Using the same tables as before.

```
SELECT CustomerName FROM Customers
EXCEPT
SELECT SupplierName FROM Suppliers;
```

Result:

CustomerName
Alice
Bob

In this example, the names "Alice" and "Bob" are customers but not suppliers, so they appear in the result set.

4. UNION ALL:

The UNION ALL operator performs the same function as the UNION operator but does not remove duplicates from the result set. It simply concatenates all rows from the different result sets.

Example: Using the same tables as before.

```
SELECT CustomerName FROM Customers
UNION ALL
SELECT SupplierName FROM Suppliers;
```

Result:

CustomerName
Alice
Bob
SupplierA
SupplierB

Difference between Set Operations and Joins

Aspect	Set Operations	Joins
Purpose	Manipulate result sets based on set theory principles.	Combine data from related tables based on specified conditions.

Data Source	Result sets of SELECT queries.	Tables that are related by common columns.
Combining Rows	Combine rows from different result sets. May remove duplicates.	Combine rows from different tables based on specified conditions.
Output Columns	Require the SELECT queries to have the same number of output columns and compatible data types.	Can combine columns from different tables, regardless of data types or column numbers.
Common Operations	UNION, INTERSECT, EXCEPT (MINUS).	INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN.
Conditional Requirements	No specific join conditions are required.	Require specified join conditions for combining data.
Handling Duplicates	UNION removes duplicates by default.	Joins do not inherently handle duplicates; it depends on the join type and data.

Usage Scenarios	Useful for combining and analysing related data from different queries or tables.	Used to retrieve and relate data from different tables based on their relationships.
Result Set Structure	Result sets may have different column names, but data types and counts must match.	Result sets can have different column names, data types, and counts.
Performance Considerations	Generally faster and less complex than joins.	Joins can be more complex and resource-intensive, especially for larger datasets.

SUB QUERIES

Subqueries, also known as nested queries or inner queries, allow you to use the result of one query (the inner query) as the input for another query (the outer query).

Subqueries are often used to retrieve data that will be used for filtering, comparison, or calculation within the context of a larger query.

They are a way to break down complex tasks into smaller, manageable steps.

Syntax:

SELECT columns

FROM table
WHERE column OPERATOR (SELECT column FROM table WHERE condition);

In this syntax:

- columns refers to the specific columns you want to retrieve from the outer query.
- table is the name of the table you're querying.
- column is the column you're applying the operator to in the outer query.
- OPERATOR is a comparison operator such as =, >, <, IN, NOT IN, etc.
- (SELECT column FROM table WHERE condition) is the subquery that provides the input for the comparison.

Example: Consider two tables: Products and Orders.

Products Table:

ProductID	ProductName	Price
1	Laptop	1000
2	Smartphone	500
3	Headphones	50

Orders Table:

OrderID	ProductID	Quantity
101	1	2
102	3	1

For Example: Retrieve the product names and quantities for orders with a total cost greater than the average price of all products.

```
SELECT ProductName, Quantity
FROM Products
WHERE Price * Quantity > (SELECT AVG(Price) FROM Products);
```

Result:

ProductName	Quantity

Laptop	2

Differences Between Subqueries and Joins:

Aspect	Subqueries	Joins
Purpose	Retrieve data for filtering, comparison, or calculation within the context of a larger query.	Combine data from related tables based on specified conditions.
Data Source	Result of one query used as input for another query.	Data from multiple related tables.
Combining Rows	Not used for combining rows; used to filter or evaluate data.	Combines rows from different tables based on specified join conditions.
Result Set Structure	Subqueries return scalar values, single-column results, or small result sets.	Joins return multi-column result sets.
Performance Considerations	Subqueries can be slower and less efficient, especially when dealing with large datasets.	Joins can be more efficient for combining data from multiple tables.
Complexity	Subqueries can be easier to understand for simple tasks or smaller datasets.	Joins can become complex, but are more suited for handling large-scale data retrieval and combination tasks.
Versatility	Subqueries can be used in various clauses: WHERE, FROM, HAVING, etc.	Joins are primarily used in the FROM clause for combining tables.